

A complete two-vertex graph defeats the clique-tree count

Disproof of Conjecture 00000003470

ziangni-sys

10 October 2026

Result. The graph K_2 has two perfect elimination orderings, but its only maximal-clique tree has one planar embedding. The claimed product formula gives one instead of two.

The source formula

Both language versions define the elimination count as the number of vertex orderings satisfying the perfect elimination successor condition. They claim that the count for chordal graphs is the product of planar-embedding counts of maximal-clique trees. We disprove this exact-count assertion; the additional logarithmic asymptotic assertion is unnecessary.

A planar embedding count is understood in the standard combinatorial sense, up to orientation-preserving topological equivalence, specified by cyclic orders of incident edges. Arbitrary geometric positions are not distinct combinatorial embeddings. The one-vertex tree used below has the same unique embedding whether reflection equivalence is also allowed or whether the root is distinguished.

The chordal graph and its actual orderings

Let $G = K_2$ on labeled vertices a, b , with the single edge ab . It is chordal: an induced cycle of length at least four would require at least four distinct vertices, which G does not have. A perfect elimination ordering is a permutation of the vertices such that the later neighbors of each vertex form a clique.

There are exactly two permutations, (a, b) and (b, a) . For either, the first vertex has the other vertex as its sole later neighbor, and the last vertex has none. A singleton and the empty set are cliques, so both permutations are valid. Hence

$$\# \text{PEO}(G) = 2.$$

This is the count of actual vertex orderings, not the count of orderings modulo graph automorphisms.

The maximal-clique tree and its embedding count

The unique maximal clique is $\{a, b\}$. Indeed, all subsets are cliques, and every proper subset extends to $\{a, b\}$. A maximal-clique tree has these maximal cliques as its vertices. There is therefore exactly one possible clique tree: one vertex labeled by $\{a, b\}$ and no edges. It is connected and acyclic. The clique-tree running-intersection condition holds because, for each original vertex, the cliques containing it form the same one-vertex connected tree.

A rotation system consists of a cyclic order of incident edge ends at every tree vertex. Here the only vertex has zero incident edge ends. There is exactly one empty cyclic order, so

exactly one rotation system. It is planar: place a single vertex in the sphere or plane, with no edges to cross. Thus

$$\# \text{Emb}(T) = 1.$$

There is only one maximal-clique tree to contribute to the source's product, and even any repeated local factors of this kind equal one. The stated product is therefore 1, whereas the true elimination-ordering count is 2. The claimed equality fails.

The same obstruction occurs on every complete labeled graph K_n with $n \geq 2$: all $n!$ vertex orderings are perfect elimination orderings, but there is again a single maximal clique and a one-vertex clique tree. Extra vertex-order factors could define a different formula, but none are part of the source's stated embedding-count product.

Formal verification

The Lean project defines the actual complete graph on `Fin 2`. It formalizes chordality by absence of induced-cycle embeddings of length at least four and proves it by the cardinality obstruction. It defines the successor-neighbor perfect elimination predicate and counts its actual valid permutations. It verifies the unique maximal clique and that every simple graph on the resulting clique-vertex type is the same one-vertex graph. It models actual local rotations as permutations of incident neighbors; since the incident sets are empty, these are precisely the unique empty cyclic orders. It computes the rotation count and the product indexed by the actual clique-tree graph type, then proves the mismatch.

Run `lake build` in `lean/`. The project pins Lean 4.19.0 and Mathlib commit

`c44e0c8ee63ca166450922a373c7409c5d26b00b`

The build prints final theorem axiom audits. There are no admitted results, custom axioms, or unchecked computations.